

计算机面试试题 RAG 智能辅导助手

基于 LangChain、Ollama、FastAPI 与 Streamlit 的本地化中文 RAG + Agent 系统

一、项目概述

本项目是一个基于 LangChain + Ollama 本地部署的中文 RAG + Agent 系统，面向计算机专业面试备考，覆盖软件工程、数据库、机器学习、大数据四个方向。系统完全本地运行，无需外部 API。

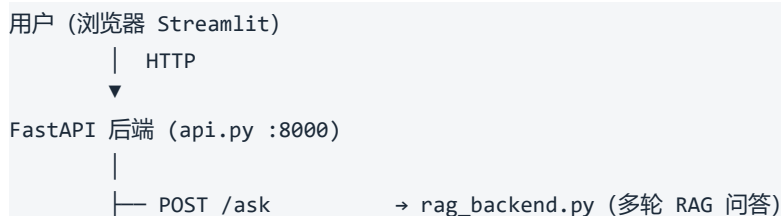


系统首页与核心交互入口

核心功能

- 混合检索：BM25 (jieba 中文分词) + 向量检索 (bge-small-zh-v1.5) 融合，召回率更高。
- 意图识别：自动识别 definition / method / comparison / general 四种问题类型，动态调整 Prompt。
- 多轮对话：ConversationBufferMemory 实现上下文记忆，支持代词追问（例如“它是什么？”）。
- Agent 路由：5 条规则路由自动分发，支持章节复习 workflow、SQL 沙箱、答案评估、Web 搜索。
- 练习模式：逐题作答，LLM 输出详细评分报告，包含评分依据、优点、不足与改进建议。
- 笔记系统：一键保存任意 AI 回答，SQLite 持久化，并按 session 隔离。

二、系统架构





关键技术选型

模块	技术选型	理由
大语言模型	Ollama + Qwen3-4B	本地推理，无需外部 API； temperature=0 保证稳定输出。
嵌入模型	BAAI/bge-small-zh-v1.5	中文语义向量，约 184MB，完全离线。
向量数据库	Chroma（持久化）	MD5 哈希版本管理，参数变更自动重建。
关键词检索	BM25 + jieba	中文无空格，jieba 切词后 BM25 才能正常召回。
Agent 路由	规则路由（非 Function Calling）	Qwen3-4B 函数调用稳定性不足， 规则路由路径固定、可追踪。
Web 框架	FastAPI + Streamlit	前后端分离，前端支持展开 Agent trace。

三、核心功能演示

3.1 章节复习 workflow

点击章节快捷按钮（如“软件工程”）后，Agent 三步串联执行：知识点总结 → 高频考点检索 → 生成模拟题，输出结构化复习报告。

模式选择

● Agent 模式 (知识补全) ⓘ

RAG 模式

仅检索本地 PDF 知识库，支持多轮追问。

章节快捷复习

软件工程

数据库

机器学习

大数据

练习模式

选择章节

软件工程 ▼

题目数量

3 0 5

开始练习

我的笔记

暂无笔记。点击下方「保存为笔记」添加。

面试题 RAG 问答助手

基于 PDF 的 RAG - BM25/向量混合检索 - 多轮追问 - 笔记系统

帮我复习一下软件工程章节

软件工程 章节复习

核心知识点

1. 非功能性：软件的“怎么做”方面，包括响应时间、安全性、可扩展性等质量属性。
2. 开闭原则：软件设计应遵循对扩展开放、对修改关闭的原则，通过抽象和接口实现功能扩展而不修改现有代码。
3. 单一职责原则：每个类或模块只负责单一功能领域，以降低耦合并提高复用性。
4. 内聚：模块内部元素的关联程度，内聚越高，模块越易维护和理解。
5. 耦合：模块之间的依赖程度，低耦合有助于提高系统模块独立性和可扩展性。

本章重点概括：本章核心在于通过开闭原则、单一职责原则、内聚和耦合等设计原则，实现软件系统的高内聚、低耦合和可维护性。

模拟面试题

Q1: 什么是软件的非功能性需求？ A1: 非功能性需求描述软件的“怎么做”，包括响应时间、安全性、可扩展性等性能和质量特性。

请输入问题，例如：CAP 定理讲的是什么？

章节复习工作流输出示例

- 核心知识点列表：5 条，含非功能性需求、开闭原则、单一职责、内聚与耦合。
- 模拟面试题：Q/A 格式，类型覆盖定义类、方法类、对比类。
- 侧边栏可切换 RAG / Agent 模式，章节快捷按钮一键触发。

3.2 答案评估

用户提交针对面试题的答案后，系统通过 eval 路由检索参考答案，LLM 输出结构化评估报告。

章节快捷复习

软件工程

数据库

机器学习

大数据

练习模式

选择章节

软件工程 ▼

题目数量

3 0 5

开始练习

我的笔记

暂无笔记。点击下方「保存为笔记」添加。

保存为笔记

针对题目：什么是软件的非功能性需求？我的答案是：是指软件产品为满足用户业务需求而必须具有且除功能需求以外的特性，包括安全性、可靠性、互操作性、健壮性等

优点

面试官准确地指出了非功能性需求是软件产品在功能需求之外必须具备的特性，并列举了安全性、可靠性、互操作性、健壮性等常见需求（注：此处“MECHANISM”为笔误，应为“特性”），这表明对非功能性需求核心概念的初步掌握。这种表述体现了良好的逻辑基础，尤其在工程实践中，这些需求（如安全性、可靠性）常被用作软件质量的关键指标，因此面试官具备较强的实践意识。

不足 / 遗漏

面试官未明确强调“软件怎么做”这一非功能性需求的核心视角（即非功能性需求聚焦于软件的内在行为、性能和质量属性，而非具体功能实现），且未提及标准答案中常见的典型例子（如响应时间、可扩展性）。此外，表述“为满足用户业务需求而必须具有”存在轻微偏差：非功能性需求是软件自身必须满足的约束条件（如性能、安全性），而非直接为用户业务需求服务；用户业务需求通常驱动功能需求，而非非功能性需求。这一表述可能引发对需求优先级的误解，需更严谨地界定其与功能需求的边界。

改进建议

非功能性需求是软件系统在功能之外的质量属性，描述软件的内在行为、性能和运行时特性，例如响应时间（系统处理请求的速度）、安全性（数据保护能力）、可扩展性（系统应对负载增长的适应性）、可能性（系统持续稳定运行的能力）等。这些需求确保软件在实际部署中满足特定的质量标准，而不涉及具体功能的实现细节。例如，一个电商系统需保证高可用性（非功能性需求），以避免因服务器故障导致用户订单丢失，而非直接定义“用户下单”这一功能。通过聚焦“软件怎么做”，非功能性需求能更精准地支撑软件的长期稳定性和用户体验，避免与功能需求混淆。

（注：本建议严格基于行业标准（如ISO/IEC 25010）和常见实践，旨在帮助面试官更精准地理解非功能性需求的本质。）

查看 Agent 执行步骤

答案评估报告示例

- 优点：识别回答中正确、完整的部分。
- 不足 / 遗漏：指出缺少的关键知识点，例如未强调“软件怎么做”视角。
- 改进建议：结合知识库参考答案给出完整补充说明。

评估报告底部提供「查看 Agent 执行步骤」和「查看检索证据」两个可折叠面板，完整还原路由决策与检索过程；右侧“已保存为笔记”说明一键保存已生效。

我的笔记

★ 优点

> 面试官准确地指出了非功能性需求是软件产

会话控制

会话 ID: b288b39d...

改进建议

非功能性需求是软件系统在功能之外的质量属性，描述软件的内在行为、性能和运行时特性，例如响应时间（系统处理请求的速度）、安全性（数据保护能力）、可扩展性（系统应对负载增长的适应性）、可靠性（系统持续稳定运行的能力）等。这些需求确保软件在实际部署中满足特定的质量标准，而不涉及具体功能的实现细节。例如，一个电商系统需保证高可用性（非功能性需求），以避免因服务器故障导致用户订单丢失，而非直接定义“用户下单”这一功能。通过聚焦“软件怎么做”，非功能性需求能更精准地支撑软件的长期稳定性和用户体验，避免与功能需求混淆。

(注：本建议严格基于行业标准（如ISO/IEC 25010）和常见实践，旨在帮助面试官更精准地理解非功能性需求的本质。)

> 查看 Agent 执行步骤

> 查看检索证据

☒ 已保存为笔记

Agent 执行步骤与检索证据

3.3 练习模式

选择章节与题目数量后点击「开始练习」，系统逐题呈现面试题并接收用户作答，提交后即时生成详细评估报告。

数据库

机器学习

大数据

练习模式

软件工程

第 1 / 3 题

结束练习

我的笔记

★ 优点

> 面试官准确地指出了非功能性需求是软件产

会话控制

会话 ID: b288b39d...

清除对话

练习模式 · 软件工程

第 1 题 / 共 3 题

你的答案

是指软件产品为满足用户业务需求而必须具有且除功能需求以外的特性，包括安全性、可靠性、互操作性、健壮性等

评估报告

评估报告：软件非功能性需求的回答

作为一位严格但友善的软件工程评估者，我将基于您的回答（“软件的非功能性是指软件产品为满足用户业务需求而必须具有且除功能需求以外的特性，例如安全性、可靠性、互操作性、健壮性等”）进行客观分析。评估依据是软件工程标准（如IEEE 830标准）和知识库中的权威定义（来自您提供的上下文：非功能性 = 软件“怎么做”（如响应时间、安全性、可扩展性））。题目要求“什么是软件的非功能性？请举例说明”，因此我会重点关注：定义的准确性、举例的完整性、表述的清晰程度。

1. 综合评分：6/10

评分依据（满分10分）：

定义准确性（3分）：您正确区分了非功能性与功能需求（“除功能需求以外的特性”），并列举了常见非功能性特性（安全性、可靠性、互操作性、健壮性），这部分得2分（满分3分）。

举例完整性（3分）：题目要求“举例说明”，但您仅列出特性名称（如“安全性”），未提供具体、可量化的例子（如响应时间、可扩展性），elicit 1分（满分3分）。：表述中“为满足用户业务需求”稍显模糊（非功能性更侧重软件的内在质量，而非直接业务需求），扣1分（满分2分）。

为什么是6分？

您的回答在核心概念上基本正确，但缺少具体例子是主要扣分点（题目明确要求“举例”）。在软件工程中，非功能性需求必须通过可测量的实例来体现（例如响应时间、安全性指标），而非仅抽象描

练习模式评分报告示例

- 综合评分（满分 10 分）附评分依据分项说明。
- 扣分点逐条说明，例如举例完整性不足、表述偏差。
- 改进方向给出参考标准答案。

四、模块详解

4.1 RAG 核心引擎（rag_backend.py）

数据处理流水线：

PDF 文件

↓ PyMuPDF 逐页提取 + 正则清洗

原始文本

↓ 按章节标题切分（软件工程 / 数据库 / 机器学习 / 大数据）

4 个章节文本块

↓ RecursiveCharacterTextSplitter

```
chunk_size=500, overlap=80, min_len=50
分隔符: 句号 / 分号 / 换行
文本 chunks (附 chapter + chunk_type 元数据)
├─ bge-small-zh-v1.5 → Chroma 向量库 (持久化 + MD5 版本管理)
└─ jieba 分词 → BM25 索引 (pickle 缓存, 启动加速)
```

混合检索

检索器	权重	算法
BM25Retriever	0.4	TF-IDF 词频统计, jieba 分词 + 停用词过滤。
Chroma 向量检索	0.6	余弦相似度, bge-small-zh-v1.5 编码。

两路各取 top-3, EnsembleRetriever 按权重融合后去重排序。

意图识别与动态 Prompt

类型	触发关键词	Prompt 指令
definition	什么是、定义、含义、概念	先给出清晰定义, 再说明关键特征。
method	如何、怎么、步骤、流程	按步骤清晰说明方法或流程。
comparison	区别、对比、优缺点、比较	分点对比异同与优缺点。
general	其他	基于参考内容准确简洁地回答。

多轮会话管理使用 ConversationBufferMemory, session_id 隔离, 最多 50 路并发, TTL 3600 秒自动清除。Chain 按 (session_id, q_type) 缓存, 避免重复构建。

4.2 Agent 路由引擎 (agent.py)

使用规则路由替代 LLM Function Calling。每条路径固定, 所有步骤均有 trace 记录。

优先级	路由	触发条件
1	review	含 “复习 / 总结 / 梳理 / 考点 / 模拟题” 等。
2	sql	含 SELECT / INSERT / CREATE 等 SQL 语句。
3	eval	含 “我的答案 / 我认为 / 评分 / 打分” 等。
4	web	含 “最新 / 趋势 / 2025 / 现状” 等时效性词。
5	rag	其他问题默认走本地知识库问答。

review 路由三步 workflow:

```

① chapter_summary → 检索 5 条文档，生成 5 个核心知识点
    ↓
② rag_search → 查询该章节高频考点
    ↓
③ generate_quiz → 生成 3 道模拟题 (Q/A 格式，含参考答案)
    ↓
组合输出：知识总结 + 模拟题 Markdown 报告

```

4.3 Agent 工具集 (tools.py)

工具	功能
rag_search	混合检索本地知识库，支持按章节过滤，返回 top-3 文档片段。
web_search	DuckDuckGo 搜索（中文区），返回最多 3 条互联网结果。
sql_executor	SQLite 内存沙箱执行 SQL，含安全拦截黑名单。
chapter_summary	三路检索 5 条文档，LLM 生成结构化知识总览。
generate_quiz	检索 3 条文档，LLM 生成 n 道 Q/A 模拟题。

4.4 SQL 沙箱 (sql_executor.py)

每次调用 `sqlite3.connect(":memory:")` 创建全新内存库，调用结束自动销毁，不写入任何本地文件。预置 6 张测试表：

表名	适用考题场景
employees / departments	薪资查询、自连接、多表 JOIN。
students / courses / enrollments	GROUP BY 聚合、子查询。
orders	日期过滤、统计分析。

五、API 接口

方法	路径	功能
POST	/ask	多轮 RAG 问答（含会话记忆）。
POST	/agent_ask	Agent 模式（路由 + trace + sources）。
POST	/clear	清除指定会话。
POST	/practice/questions	生成练习题目列表（优先走缓存）。
POST	/practice/evaluate	评估练习答案。
POST	/notes/save	保存笔记。
GET	/notes	查询笔记列表。
DELETE	/notes/{note_id}	删除笔记。

/agent_ask 返回字段: answer / question_type / route / used_web / sql_result / sources /

trace

六、部署指南

环境要求: Python 3.9+, Ollama, 内存 $\geq 8\text{GB}$, 磁盘 $\geq 4\text{GB}$ 。

```
# 1. 安装依赖
pip install -r requirements.txt

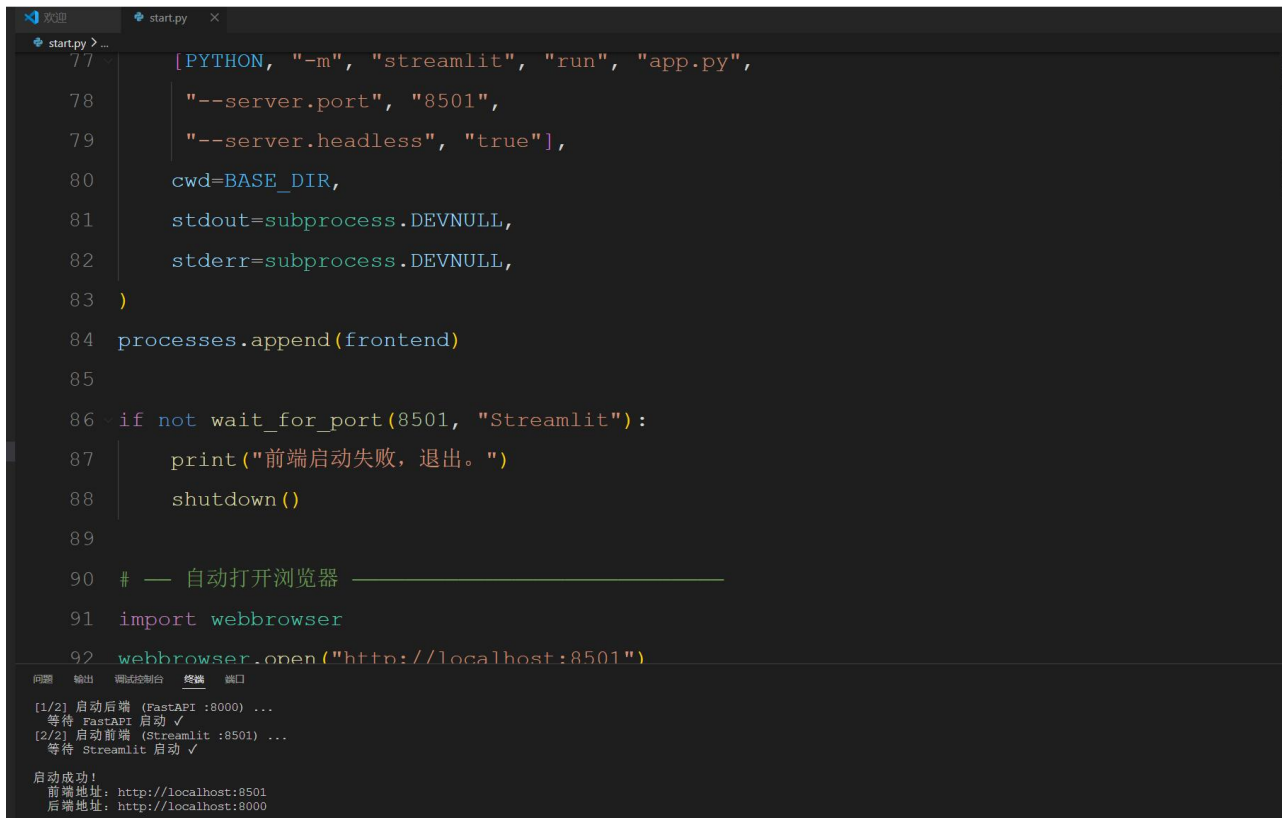
# 2. 下载嵌入模型
huggingface-cli download BAAI/bge-small-zh-v1.5 --local-dir ./bge-small-zh-v1.5

# 3. 启动 Ollama
ollama serve
ollama pull qwen3:4b

# 4. 放入 PDF (data/章节面试题.pdf, 需含四个章节标题)

# 5. 一键启动
python start.py
```

浏览器自动打开 <http://localhost:8501>, Ctrl+C 停止所有服务。下图为 IDE 中的项目文件结构与终端启动日志, FastAPI (:8000) 与 Streamlit (:8501) 均已成功拉起。



```
start.py
77 [PYTHON, "-m", "streamlit", "run", "app.py",
78     "--server.port", "8501",
79     "--server.headless", "true"],
80     cwd=BASE_DIR,
81     stdout=subprocess.DEVNULL,
82     stderr=subprocess.DEVNULL,
83 )
84 processes.append(frontend)
85
86 if not wait_for_port(8501, "Streamlit"):
87     print("前端启动失败, 退出。")
88     shutdown()
89
90 # — 自动打开浏览器 —————
91 import webbrowser
92 webbrowser.open("http://localhost:8501")

[1/2] 启动后端 (FastAPI :8000) ...
等待 FastAPI 启动 ✓
[2/2] 启动前端 (Streamlit :8501) ...
等待 Streamlit 启动 ✓

启动成功!
前端地址: http://localhost:8501
后端地址: http://localhost:8000
```

项目文件结构与服务启动日志

首次启动耗时约 1~3 分钟（PDF 解析 → 向量库构建 → BM25 索引），后续启动参数未变更时复用缓存，耗时约 10~20 秒。